UNIVERSIDADE FEDERAL DE MATO GROSSO

Organização de Computadores · Prof. Linder Cândido

Explicação Detalhada do Código

Relógio Digital em Assembly RISC-V Bare Metal

Raissa Cavalcanti e Luiz Torres

Março de 2026

1. Glossário de Termos Técnicos

Todos os termos técnicos utilizados neste trabalho são definidos abaixo em ordem de relevância para a compreensão do código.

Termo	Definição
`Bare Metal`	Execução direta sobre o hardware, sem sistema operacional ou qualquer camada de software intermediária.
`RISC-V`	Arquitetura de conjunto de instruções (ISA) open-source baseada no paradigma RISC (Reduced Instruction Set Computer). Projetada para ser simples, modular e extensível.
`RV32IMA`	Variante do RISC-V: 32 bits, com extensão I (inteiros base), M (multiplicação e divisão) e A (operações atômicas).
`Assembly / ASM`	Linguagem de programação de baixo nível onde cada instrução corresponde diretamente a uma operação do processador.
`MMIO`	Memory-Mapped I/O — periféricos cujos registradores são acessados como endereços de memória comuns.
`UART`	Universal Asynchronous Receiver/Transmitter - circuito de comunicação serial assíncrona. Converte dados em uma sequência de bits transmitidos por um único fio.
`NS16550`	Modelo clássico de chip UART, amplamente compatível e emulado pelo QEMU na plataforma virt.
`PLIC`	Platform-Level Interrupt Controller - controlador central de interrupções externas do RISC-V. Gerencia prioridades, habilitações e entrega de IRQs para os harts.
`CLINT`	Core-Local Interruptor - fornece o timer de hardware (mtime/mtimecmp) e interrupções de software por hart.
`mtime`	Registrador de 64 bits do CLINT que conta continuamente na frequência do clock do sistema (10 MHz no QEMU virt). Somente leitura.
`mtimecmp`	Registrador de 64 bits do CLINT. Quando mtime >= mtimecmp, uma Machine Timer Interrupt é gerada para o hart correspondente.
`CSR`	Control and Status Register - registradores especiais do processador acessíveis apenas por instruções CSR (csrr, csrw, csrs, csrc).
`mtvec`	CSR Machine Trap Vector - endereço para onde a CPU salta ao receber qualquer trap (interrupção ou exceção).
`mcause`	CSR Machine Cause - descreve o motivo da última trap. Bit 31=1 indica interrupção; bits 30:0 indicam o código da causa.
`mstatus`	CSR Machine Status - contém o bit MIE (bit 3) que habilita/desabilita globalmente todas as interrupções.
`mie`	CSR Machine Interrupt Enable - bits individuais que habilitam cada tipo de interrupção: bit 7=MTIE (timer), bit 11=MEIE (external).
`mret`	Machine Return - instrução de retorno de trap. Restaura PC←mepc e reabilita interrupções conforme mstatus.MPIE.
`ISR`	Interrupt Service Routine - rotina de tratamento de uma interrupção específica.
`wfi`	Wait For Interrupt - instrução que suspende o hart até a chegada de uma interrupção, economizando energia e evitando busy-wait.
`hart`	Hardware Thread - núcleo de execução do RISC-V. QEMU virt usa um único hart (hart0) no modo single-core.

`Callee-saved`	Registradores que uma função DEVE preservar ao retornar (s0–s11, ra). Se usados, devem ser salvos e restaurados na pilha.
`Caller-saved`	Registradores que podem ser sobrescritos por qualquer função (t0–t6, a0–a7). O chamador deve salvar se precisar preservá-los.
`Stack Pointer (sp)`	Registrador x2 - aponta para o topo atual da pilha. A pilha RISC-V cresce de endereços altos para baixos.
`ra`	Return Address - registrador x1 que armazena o endereço de retorno de uma chamada de função (salvo por jal).
`jal rd, label`	Jump And Link - salta para label e salva PC+4 em rd. Usado para chamadas de função (jal ra, func).
`jr ra`	Jump Register ra - retorna de uma função saltando para o endereço em ra.
`la rd, label`	Load Address - pseudoinstrução que carrega o endereço de um símbolo em rd.
`li rd, imm`	Load Immediate - pseudoinstrução que carrega um valor imediato em rd.
`lw / sw`	Load Word / Store Word - lê/escreve 4 bytes de/para memória.
`lb / sb`	Load Byte / Store Byte - lê/escreve 1 byte de/para memória.
`lbu`	Load Byte Unsigned - lê 1 byte e zera os 24 bits superiores (sem extensão de sinal).
`slli rd, rs, n`	Shift Left Logical Immediate - desloca rs n bits à esquerda. Equivale a multiplicar por 2^n.
`sltu rd, rs1, rs2`	Set Less Than Unsigned - rd=1 se rs1 < rs2 (sem sinal), rd=0 caso contrário. Usado para detectar carry.
`csrr rd, csr`	CSR Read - lê o valor do CSR para rd.
`csrw csr, rs`	CSR Write - escreve rs no CSR.
`csrs csr, rs`	CSR Set bits - seta os bits indicados por rs no CSR (OR bit a bit).
`csrc csr, rs`	CSR Clear bits - limpa os bits indicados por rs no CSR (AND NOT).
`.equ nome, val`	Diretiva: define uma constante simbólica (equivalente ao #define do C).
`.section`	Diretiva: inicia uma nova seção no binário (.text, .data, .bss).
`.align n`	Diretiva: alinha o endereço atual a um múltiplo de 2^n bytes.
`.space n`	Diretiva: reserva n bytes não inicializados.
`.byte val`	Diretiva: define 1 byte com valor inicial.
`.ascii str`	Diretiva: declara uma string de bytes (sem null terminator automático — \0 deve ser explícito).
`.global sym`	Diretiva: torna o símbolo visível para o linker (necessário para _start).
`Linker Script`	Arquivo .ld que instrui o linker sobre onde posicionar cada seção do binário na memória.
`PLIC Claim`	Leitura do registrador PLIC_CLAIM - retorna o ID da IRQ pendente de maior prioridade e a marca como "em atendimento".
`PLIC Complete`	Escrita do IRQ ID no PLIC_CLAIM - informa ao PLIC que a IRQ foi atendida e pode ser entregue novamente.

2. Arquivo clock.S - Código Assembly Principal

2.1 Cabeçalho e opção do assembler

Instrução / Diretiva	Explicação
`.option norvc`	Desabilita a extensão de instruções comprimidas de 16 bits (RVC). Garante que todas as instruções sejam de 32 bits, evitando problemas de alinhamento no trap_handler e simplificando o debug.

2.2 Constantes - UART NS16550

O NS16550 é byte-addressable: cada registrador ocupa 1 byte. Os offsets são somados ao UART_BASE (0x10000000):

Instrução / Diretiva	Explicação
`.equ UART_BASE, 0x10000000`	Endereço físico base da UART0 na plataforma QEMU virt. Todo acesso à UART usa este endereço como base.
`.equ UART_RHR,  0`	Receive Holding Register - byte 0 do UART_BASE. Leitura retorna o byte recebido mais recente (válido quando LSR bit0=1).
`.equ UART_THR,  0`	Transmit Holding Register - mesmo offset do RHR (byte 0). Escrita envia um byte pela UART.
`.equ UART_IER,  1`	Interrupt Enable Register - byte 1. Bit 0 (ERBFI): habilita IRQ ao receber dados. DEVE ser 1 (byte offset, não word offset 4).
`.equ UART_LSR,  5`	Line Status Register - byte 5. Bit 0=dado disponível para leitura. Bit 5=TX Holding Empty (pode transmitir).
`.equ LSR_DR,    0x01`	Máscara do bit 0 do LSR: Data Ready. Indica que há um byte no RHR aguardando leitura.
`.equ LSR_THRE,  0x20`	Máscara do bit 5 do LSR: TX Holding Register Empty. Indica que a UART está pronta para receber o próximo byte a transmitir.

2.3 Constantes - PLIC

O PLIC do QEMU virt reserva endereços específicos para cada hart e modo de privilégio. UART0 usa a IRQ de número 10:

Instrução / Diretiva	Explicação
`.equ PLIC_PRIORITY, 0x0C000000`	Base dos registradores de prioridade. O registrador da IRQ N está em PLIC_PRIORITY + N*4. Prioridade 0 = desabilitada; 1-7 = habilitada (maior número = maior prioridade).
`.equ PLIC_ENABLE, 0x0C002000`	Registrador de habilitação para hart0 M-mode. Cada bit corresponde a uma IRQ: bit N controla a IRQ N. Múltiplos de 32 bits cobrem 32 IRQs por registrador.
`.equ PLIC_THRESHOLD, 0x0C200000`	Threshold de prioridade do hart0 M-mode. Apenas IRQs com prioridade > threshold são entregues ao hart. Zero aceita qualquer IRQ de prioridade >= 1.

`.equ PLIC_CLAIM, 0x0C200004`	Registrador de claim/complete do hart0 M-mode. Leitura = claim (retorna ID da IRQ pendente). Escrita = complete (confirma atendimento).
`.equ UART_IRQ, 10`	Número da IRQ da UART0 na plataforma QEMU virt. Definido pela especificação da plataforma virt do QEMU.

2.4 Constantes - CLINT

O CLINT usa endereços absolutos porque o GNU Assembler não aceita expressões compostas como operando de memória (ex: CLINT_BASE+0xBFF8 seria inválido em lw):

Instrução / Diretiva	Explicação
`.equ MTIME_LO, 0x0200BFF8`	Endereço da word baixa (bits 31:0) do contador mtime de 64 bits. Incrementa continuamente a 10 MHz no QEMU virt.
`.equ MTIME_HI, 0x0200BFFC`	Endereço da word alta (bits 63:32) do contador mtime. As duas words formam um contador de 64 bits.
`.equ MTIMECMP_LO, 0x02004000`	Endereço da word baixa do comparador mtimecmp. IRQ de timer gerada quando mtime >= mtimecmp.
`.equ MTIMECMP_HI, 0x02004004`	Endereço da word alta do comparador mtimecmp.
`.equ TIMER_HZ, 10000000`	10.000.000 ticks = 1 segundo. Somado ao mtime atual para programar o próximo disparo exatamente 1 segundo depois.

2.5 Constantes de mcause e CSR bits

Instrução / Diretiva	Explicação
`.equ CAUSE_TIMER, 0x80000007`	Machine Timer Interrupt: bit 31=1 (é interrupção, não exceção) + código 7 (timer). Valor de 32 bits completo para comparação direta com mcause.
`.equ CAUSE_EXTERNAL, 0x8000000B`	Machine External Interrupt: bit 31=1 + código 11 (interrupção externa / PLIC). Gerada quando o PLIC entrega uma IRQ ao hart.
`.equ MIE_MTIE, (1 << 7)`	Bit 7 do CSR mie: Machine Timer Interrupt Enable. Setá-lo permite que interrupções de timer alcancem o hart.
`.equ MIE_MEIE, (1 << 11)`	Bit 11 do CSR mie: Machine External Interrupt Enable. Setá-lo permite que o PLIC entregue IRQs ao hart.
`.equ MSTATUS_MIE, (1 << 3)`	Bit 3 do CSR mstatus: Machine Interrupt Enable global. Deve ser setado APÓS configurar todos os periféricos e antes do wfi.

2.6 Seção .bss - Pilha de 4 KB

Instrução / Diretiva	Explicação
`.section .bss`	Início da seção de dados não inicializados. O linker posiciona esta seção após .data e zera o conteúdo.
`.align 4`	Alinha o endereço atual a 2^4 = 16 bytes. Garante que a pilha comece em endereço bem alinhado.
`.space 4096`	Reserva 4096 bytes (4 KB) para a pilha. Espaço suficiente para frames aninhados de ISR + subfunções.
`stack_top:`	Label que aponta para o byte imediatamente APÓS os 4 KB reservados. A pilha RISC-V cresce para baixo: SP começa aqui e diminui a cada push.

2.7 Seção .data - Variáveis e Strings

Instrução / Diretiva	Explicação
`.section .data`	Início da seção de dados inicializados. Carregada do binário para a RAM na inicialização.
`.align 2`	Alinha a 2^2 = 4 bytes. Garante acesso word-aligned às variáveis.
`g_horas:  .byte 0`	Variável global de 1 byte: valor das horas (0–23). Inicializada em 0.
`g_minutos:  .byte 0`	Variável global de 1 byte: valor dos minutos (0–59).
`g_segundos: .byte 0`	Variável global de 1 byte: valor dos segundos (0–59).
`g_rxbuf:  .space 16`	Buffer de recepção de 16 bytes para acumular caracteres digitados antes do Enter.
`g_rxlen:  .byte 0`	Número de bytes válidos atualmente armazenados em g_rxbuf.
`str_banner: .ascii ...\0`	String do banner inicial. \r\n = Carriage Return + Line Feed. \0 = terminador nulo obrigatório para uart_puts.
`str_ok: .ascii ...\0`	Mensagem de confirmação após ajuste de hora bem-sucedido.
`str_err: .ascii ...\0`	Mensagem de erro exibida quando o formato do comando está incorreto.

2.8 Função _start - Ponto de Entrada e Inicialização

O _start é o primeiro código executado pelo processador. Configura tudo que é necessário antes de entrar no loop principal:

Instrução / Diretiva	Explicação
`.section .text`	Início da seção de código executável.
`.global _start`	Torna _start visível ao linker como ponto de entrada. Sem isso, o linker não sabe onde o programa começa.
`la sp, stack_top`	PASSO 1: carrega o endereço de stack_top no registrador sp (x2). A partir daqui qualquer chamada jal pode usar a pilha.
`la t0, trap_handler`	PASSO 2a: carrega o endereço do trap_handler em t0.
`csrw mtvec, t0`	PASSO 2b: escreve o endereço no CSR mtvec. A partir daqui, toda trap (interrupção ou exceção) faz a CPU saltar para trap_handler.
`li t0, UART_BASE`	PASSO 3a: t0 = 0x10000000 (endereço base da UART).
`li t1, 1`	PASSO 3b: t1 = 1 (habilita bit 0 do IER: ERBFI = Enable Received Buffer Full Interrupt).
`sb t1, UART_IER(t0)`	PASSO 3c: escreve 1 no byte 1 do UART_BASE. A UART agora gera uma IRQ sempre que receber um byte.
`li t0, PLIC_PRIORITY`	PASSO 4a: t0 = endereço base dos registradores de prioridade do PLIC.
`li t1, UART_IRQ`	PASSO 4b: t1 = 10 (número da IRQ da UART0).
`slli t1, t1, 2`	PASSO 4c: t1 = 10 * 4 = 40 (offset = IRQ * 4, pois cada reg. de prioridade ocupa 4 bytes).
`add t0, t0, t1`	PASSO 4d: t0 = PLIC_PRIORITY + 40 = endereço do registrador de prioridade da IRQ 10.
`li t2, 1 / sw t2, 0(t0)`	PASSO 4e-f: escreve prioridade = 1 para a IRQ 10. Prioridade 0 = desabilitada; qualquer valor > 0 habilita.
`li t0, PLIC_ENABLE`	PASSO 5a: endereço do registrador de habilitação das IRQs do hart0 M-mode.
`li t1, (1 << UART_IRQ)`	PASSO 5b: t1 = 1 << 10 = 0x400. Bit 10 corresponde à IRQ 10.
`sw t1, 0(t0)`	PASSO 5c: habilita a IRQ 10 no PLIC. Agora o PLIC pode entregar a IRQ da UART ao hart.
`li t0, PLIC_THRESHOLD / sw zero`	PASSO 6: threshold = 0. O PLIC entregará qualquer IRQ com prioridade >= 1.
`jal timer_set`	PASSO 7: chama timer_set para programar o primeiro disparo do timer (mtime + 10.000.000 = daqui a 1 segundo).
`li t0,(MIE_MTIE|MIE_MEIE) / csrs mie,t0`	PASSO 8: seta os bits 7 e 11 no CSR mie. Habilita interrupções de timer E externas individualmente.
`li t0, MSTATUS_MIE / csrs mstatus,t0`	PASSO 9: seta o bit 3 do CSR mstatus. Liga o enable global de interrupções. A partir daqui o processador pode ser interrompido.
`la a0, str_banner / jal uart_puts`	PASSO 10a: exibe o banner de boas-vindas pela UART.
`jal display_time`	PASSO 10b: exibe 00:00:00 como hora inicial.

2.9 Loop Principal

Instrução / Diretiva	Explicação
`main_loop:`	Label de início do loop infinito.
`wfi`	Wait For Interrupt: suspende o hart até a chegada de uma interrupção. CPU fica em estado idle, consumindo energia mínima. Substitui o busy-wait.
`j main_loop`	Após retornar da ISR (via mret), salta de volta para wfi. O loop nunca executa lógica — apenas aguarda.

2.10 Função timer_set - Programação do Timer

Calcula o próximo disparo como mtime_atual + 10.000.000 e escreve no mtimecmp. Implementa leitura segura de 64 bits e escrita atômica:

Instrução / Diretiva	Explicação
`li t0, MTIME_LO / li t1, MTIME_HI`	Carrega os endereços das duas words do mtime (LOW=0x0200BFF8, HIGH=0x0200BFFC).
`ts_retry:`	Label de reentrada para o caso de carry detectado durante a leitura.
`lw t3, 0(t1)`	1ª leitura do HIGH do mtime — salva em t3.
`lw t2, 0(t0)`	Leitura do LOW do mtime — salva em t2.
`lw t4, 0(t1)`	2ª leitura do HIGH do mtime — salva em t4.
`bne t3, t4, ts_retry`	Se t3 != t4, o HIGH mudou durante a leitura do LOW (carry ocorreu). Repete o processo.
`li t4, TIMER_HZ`	Carrega 10.000.000 em t4.
`add t5, t2, t4`	cmp_lo = mtime_lo + TIMER_HZ. Resultado pode ter overflow (carry para a parte alta).
`sltu t4, t5, t2`	Detecta carry: se t5 < t2 (unsigned), houve overflow. t4 = 1 se carry, 0 se não.
`add t4, t3, t4`	cmp_hi = mtime_hi + carry. Propaga o carry para a parte alta do novo comparador.
`li t6, 0x7FFFFFFF`	Valor máximo para bloquear o timer temporariamente durante a escrita.
`sw t6, 0(t1)`	Escreve HIGH=MAX no mtimecmp. Enquanto HIGH está inflado, mtime jamais alcança mtimecmp, evitando disparo espúrio.
`sw t5, 0(t0)`	Escreve o LOW definitivo. Agora a parte baixa está correta.
`sw t4, 0(t1)`	Escreve o HIGH definitivo. A escrita está completa e o timer é reprogramado de forma segura.
`jr ra`	Retorna ao chamador.

2.11 trap_handler - Vetor de Interrupções

Único ponto de entrada para todas as traps. Salva todo o estado, identifica a causa e despacha para a ISR correta:

Instrução / Diretiva	Explicação
`.align 4`	Alinha trap_handler a 2^4 = 16 bytes. Obrigatório para o modo Direct do mtvec (bits [1:0] = 0b00).
`addi sp, sp, -80`	Reserva 80 bytes na pilha: 20 registradores x 4 bytes cada.
`sw ra, 0(sp)`	Salva ra (endereço de retorno do código interrompido). Obrigatório — jal dentro da ISR sobrescreveria ra.
`sw t0..t6, 4..28(sp)`	Salva os 7 registradores temporários t0–t6. Qualquer função chamada na ISR pode sobrescrevê-los.
`sw a0..a7, 32..60(sp)`	Salva os 8 registradores de argumento/retorno. Usados por uart_putc, uart_puts, etc.

`sw s0, 64(sp)`	Salva s0 (callee-saved). Será usado para armazenar o IRQ ID do PLIC entre Claim e Complete.
`sw s1, 68(sp)`	Salva s1. Será usado para armazenar o byte recebido da UART, sobrevivendo a chamadas jal.
`sw s2, 72(sp)`	Salva s2. Usado em dec2 para preservar o valor original durante as duas chamadas uart_putc.
`csrr t0, mcause`	Lê o CSR mcause. Valor 0x80000007 = timer; 0x8000000B = external (PLIC/UART).
`li t1, CAUSE_TIMER / beq t0,t1,isr_timer`	Se mcause == timer, salta para isr_timer.
`li t1, CAUSE_EXTERNAL / beq t0,t1,isr_external`	Se mcause == external, salta para isr_external.
`j trap_exit`	Qualquer outro valor (exceção desconhecida): ignora e retorna normalmente.

ISR do Timer (isr_timer)

Instrução / Diretiva	Explicação
`jal timer_set`	Reprograma mtimecmp para o próximo segundo. Se não fizer isso, o timer não disparará mais.
`jal tick_second`	Incrementa os segundos. Se atingir 60, zera e incrementa os minutos. Se minutos atingirem 60, incrementa horas. Se horas atingirem 24, zera tudo.
`jal display_time`	Envia \rHH:MM:SS pela UART, sobrescrevendo o valor anterior na mesma linha do terminal.
`j trap_exit`	Salta para o código de restauração de contexto e mret.

ISR Externa - UART via PLIC (isr_external)

Instrução / Diretiva	Explicação
`li t0, PLIC_CLAIM / lw s0, 0(t0)`	PLIC Claim: lê o registrador. Retorna o ID da IRQ pendente e a marca como em atendimento. Salvo em s0 (callee-saved) para sobreviver às chamadas jal seguintes.
`li t1, UART_IRQ / bne s0,t1,isr_done`	Verifica se a IRQ é da UART (ID=10). Se não for, pula o processamento e vai direto para o Complete.
`li t0, UART_BASE / lb s1, UART_RHR(t0)`	Lê o byte recebido do Receive Holding Register. Salvo em s1 (callee-saved).
`mv a0, s1 / jal uart_putc`	Ecoa o byte: envia de volta ao terminal para que o usuário veja o que está digitando.
`li t0,'\r' / beq s1,t0,isr_eol`	Se o byte é CR (Enter), vai processar o comando completo.
`li t0,'\n' / beq s1,t0,isr_eol`	Se o byte é LF (Enter em alguns sistemas), também processa.
`la t0, g_rxlen / lbu t1, 0(t0)`	Lê o comprimento atual do buffer de recepção.
`li t2, 15 / bge t1,t2,isr_done`	Se o buffer está cheio (>= 15 bytes), descarta o caractere.
`la t2, g_rxbuf / add t2,t2,t1 / sb s1,0(t2)`	Armazena o byte na próxima posição livre do buffer.
`addi t1,t1,1 / sb t1,0(t0)`	Incrementa e salva o novo comprimento do buffer.

`isr_eol: null-termina + jal parse_command`	Fecha a string com \0 e chama parse_command para processar o comando.
`sb zero, 0(t0)`	Reseta g_rxlen = 0 para receber o próximo comando.
`li t0,PLIC_CLAIM / sw s0,0(t0)`	PLIC Complete: devolve o IRQ ID (s0 original) ao PLIC, sinalizando que a IRQ foi tratada.

2.12 Restauração de Contexto e mret

Instrução / Diretiva	Explicação
`lw ra,0(sp) ... lw s2,72(sp)`	Restaura os 19 registradores na ordem inversa ao salvamento. Cada valor volta exatamente ao registrador correspondente.
`addi sp, sp, 80`	Libera o frame da pilha, desfazendo o addi sp,sp,-80 inicial.
`mret`	Machine Return: restaura PC←mepc (onde o código foi interrompido), restaura o nível de privilégio e reabilita interrupções conforme mstatus.MPIE. A CPU retorna ao wfi.

2.13 tick_second - Incremento do Relógio

Instrução / Diretiva	Explicação
`la t0, g_segundos / lbu t1, 0(t0)`	Carrega o endereço e lê o valor atual dos segundos (byte unsigned, 0–255 sem extensão de sinal).
`addi t1, t1, 1`	Incrementa 1 segundo.
`li t2, 60 / blt t1,t2,ts_seg`	Se segundos ainda < 60, pula direto para salvar o novo valor.
`li t1, 0`	Segundos chegaram a 60: zera.
`la t0, g_minutos / lbu t2, 0(t0) / addi t2,t2,1`	Lê e incrementa os minutos.
`li t3, 60 / blt t2,t3,ts_min`	Se minutos < 60, salva sem incrementar horas.
`li t2, 0`	Minutos chegaram a 60: zera.
`la t0, g_horas / lbu t3, 0(t0) / addi t3,t3,1`	Lê e incrementa as horas.
`li t4, 24 / blt t3,t4,ts_hora`	Se horas < 24, salva normalmente.
`li t3, 0`	Horas chegaram a 24 (meia-noite): zera. 23:59:59 → 00:00:00.
`ts_hora/ts_min/ts_seg: sb ...`	Salva os valores atualizados nos endereços das variáveis globais correspondentes.

2.14 display_time - Formata e Exibe HH:MM:SS

Instrução / Diretiva	Explicação
`li a0,'\r' / jal uart_putc`	Envia Carriage Return (código ASCII 13): retorna o cursor ao início da linha atual, sem avançar para a próxima. A próxima exibição sobrescreve a anterior.
`la t0, g_horas / lbu a0, 0(t0) / jal dec2`	Lê o valor das horas e chama dec2 para imprimir como dois dígitos (ex: 9 → "09").
`li a0,':' / jal uart_putc`	Imprime o caractere ':' como separador entre HH e MM.
`(repete para minutos e segundos)`	Mesma sequência: lê a variável, chama dec2, imprime ':'.
`li a0,' ' / jal uart_putc (×2)`	Envia dois espaços para apagar eventuais caracteres extras que possam ter ficado da linha anterior.

2.15 dec2 - Converte Número em Dois Dígitos ASCII

Instrução / Diretiva	Explicação
`addi sp,sp,-8 / sw ra,0(sp) / sw s2,4(sp)`	Salva ra (para poder usar jal) e s2 (para preservar o valor de entrada entre as duas chamadas uart_putc).
`mv s2, a0`	Copia o valor recebido para s2 (callee-saved). a0 será sobrescrito pelas chamadas jal.
`li t0, 10 / div t1, s2, t0`	Divisão inteira: t1 = s2 / 10 = dezena. Ex: 35 / 10 = 3.
`rem t2, s2, t0`	Resto da divisão: t2 = s2 % 10 = unidade. Ex: 35 % 10 = 5.
`li t3,'0' / add a0,t1,t3 / jal uart_putc`	Converte a dezena para ASCII somando o código de '0' (48). Ex: 3 + 48 = '3'. Envia pela UART.
`li t3,'0' / add a0,t2,t3 / jal uart_putc`	Mesma conversão para a unidade. Ex: 5 + 48 = '5'. Envia pela UART.
`lw ra,0(sp) / lw s2,4(sp) / addi sp,sp,8 / jr ra`	Restaura contexto e retorna.

2.16 parse_command - Interpreta "T HH:MM:SS"

Valida o comando byte a byte antes de aplicar qualquer alteração. Usa s0=HH, s1=MM, s2=SS para armazenar os valores validados:

Instrução / Diretiva	Explicação
`addi sp,sp,-16 / sw ra,s0,s1,s2`	Salva ra e s0/s1/s2 na pilha (serão reutilizados para armazenar os valores validados).
`la t0, g_rxbuf`	t0 aponta para o início do buffer com o comando recebido.
`lbu t1,0(t0) / li t2,'T' / bne t1,t2,pc_err`	Posição 0 deve ser o caractere 'T'. Se não for, vai para o tratamento de erro.
`lbu t1,1(t0) / li t2,' ' / bne t1,t2,pc_err`	Posição 1 deve ser um espaço.
`lbu a0,2(t0) / lbu a1,3(t0) / jal p2d`	Lê os dois caracteres das horas (posições 2 e 3) e chama p2d para converter.
`li t1,-1 / beq a0,t1,pc_err`	p2d retorna -1 se algum char não era dígito. Neste caso vai para o erro.
`li t1,24 / bge a0,t1,pc_err`	Valida o range: horas deve ser < 24. Valores 24–99 são inválidos.
`mv s0, a0`	Salva o valor validado das horas em s0 (callee-saved).
`(repete para ':' MM ':' SS)`	Mesma lógica: verifica separadores ':' e converte MM e SS com p2d, validando ranges.
`csrc mstatus, t0 (MIE=0)`	Desabilita interrupções globais para garantir que as 3 variáveis sejam atualizadas atomicamente (sem risco de isr_timer ler valores parcialmente atualizados).
`sb s0,0(g_horas) / sb s1,0(g_minutos) / sb s2,0(g_segundos)`	Aplica os novos valores de hora, minuto e segundo nas variáveis globais.
`csrs mstatus, t0 (MIE=1)`	Reabilita interrupções.
`jal uart_puts (str_ok) / jal display_time`	Exibe confirmação e a nova hora imediatamente.

`pc_err: jal uart_puts (str_err)`	Em caso de erro, exibe a mensagem de erro.

2.17 p2d - Dois Chars ASCII para Inteiro

Instrução / Diretiva	Explicação
`li t0,'0' / li t1,'9'`	Limites do range ASCII dos dígitos decimais: '0'=48, '9'=57.
`blt a0,t0,p2d_err / bgt a0,t1,p2d_err`	Valida a dezena: se menor que '0' ou maior que '9', retorna -1 (inválido).
`blt a1,t0,p2d_err / bgt a1,t1,p2d_err`	Valida a unidade da mesma forma.
`sub t2,a0,t0 / sub t3,a1,t0`	Converte de ASCII para número: '5' - '0' = 5.
`li t4,10 / mul t2,t2,t4`	Multiplica a dezena por 10.
`add a0,t2,t3 / jr ra`	a0 = dezena*10 + unidade. Retorna o valor final (0–99) em a0.
`p2d_err: li a0,-1 / jr ra`	Retorna -1 como código de erro para indicar input inválido.

2.18 uart_putc - Envia Um Byte pela UART

Instrução / Diretiva	Explicação
`li t0, UART_BASE`	Carrega o endereço base da UART.
`uart_tx_wait:`	Loop de espera pela disponibilidade do transmissor.
`lbu t1, UART_LSR(t0)`	Lê o Line Status Register (byte 5 da UART).
`andi t1, t1, LSR_THRE`	Isola o bit 5: TX Holding Register Empty.
`beqz t1, uart_tx_wait`	Se o bit é 0 (transmissor ocupado), aguarda em loop.
`sb a0, UART_THR(t0)`	Transmissor livre: escreve o byte no THR. A UART inicia a transmissão serial.
`jr ra`	Retorna ao chamador.

2.19 uart_puts - Envia String Null-Terminated

Instrução / Diretiva	Explicação
`addi sp,sp,-8 / sw ra,0(sp) / sw s0,4(sp)`	Salva ra (para poder usar jal) e s0 (para preservar o ponteiro da string entre iterações).
`mv s0, a0`	Copia o ponteiro para s0 (callee-saved). a0 será sobrescrito por uart_putc.
`uart_puts_loop: lbu a0, 0(s0)`	Lê o próximo byte da string (unsigned para não confundir \0 com -1).
`beqz a0, uart_puts_done`	Se o byte é \0 (terminador nulo), encerra o loop.
`jal uart_putc / addi s0,s0,1`	Envia o byte e avança o ponteiro para o próximo caractere.
`j uart_puts_loop`	Repete o loop.

`uart_puts_done: lw ra / lw s0 / addi sp / jr ra`	Restaura contexto e retorna.

3. Arquivo link.ld - Linker Script

Define como as seções do binário compilado são organizadas na memória física do sistema:

Instrução / Diretiva	Explicação
`OUTPUT_ARCH(riscv)`	Informa ao linker que o alvo é RISC-V (influencia o formato do ELF gerado).
`ENTRY(_start)`	Define _start como símbolo de entrada. O QEMU usa este endereço como PC inicial após carregar o kernel.
`RAM (rwx) ORIGIN=0x80000000`	Define a região RAM a partir de 0x80000000 com permissões de leitura, escrita e execução. O QEMU carrega o kernel neste endereço.
`.text > RAM`	Seção de código posicionada no início da RAM. Primeiro bloco executado.
`. = ALIGN(4)`	Garante que as seções seguintes começam em endereços alinhados a 4 bytes (word-aligned).
`.rodata > RAM`	Dados somente leitura (constantes, strings literais). Posicionado após .text.
`.data > RAM`	Dados inicializados (variáveis globais com valores iniciais). Posicionado após .rodata.
`.bss > RAM`	Dados não inicializados (pilha). O linker garante que esta área seja zerada.
`__bss_start / __bss_end`	Símbolos exportados que marcam início e fim do .bss. Úteis para zero-init manual se necessário.

4. Arquivo Makefile

Automatiza o processo de compilação e execução com regras claras:

Instrução / Diretiva	Explicação
`CROSS ?= riscv64-unknown-elf`	Prefixo do toolchain. O operador ?= permite sobrescrever: make CROSS=outro-prefixo.
`CC = $(CROSS)-gcc`	Compilador/assembler GCC para RISC-V. Usado para montar o .S e ligar o .elf.
`OBJDUMP = $(CROSS)-objdump`	Ferramenta de disassembly para inspecionar o binário gerado.
`-march=rv32ima`	Define a ISA alvo: RV32 com extensões I (base), M (mul/div), A (atômicas).
`-mabi=ilp32`	Application Binary Interface de 32 bits: int, long e pointers têm 32 bits.
`-nostdlib`	Não linka nenhuma biblioteca padrão C (libc, etc.).
`-nostartfiles`	Não inclui o arquivo crt0.o (startup C). Nosso _start é o único ponto de entrada.
`-T link.ld`	Usa o linker script personalizado ao invés do padrão.
`-lgcc`	Linka a biblioteca GCC runtime que implementa operações como mul e div em software quando necessário.

`-machine virt`	Plataforma QEMU: 'virt' tem UART0, PLIC e CLINT nos endereços que usamos.
`-nographic`	Redireciona a saída de vídeo para a serial (UART0 = terminal). Necessário para execução via Docker.
`-bios none`	Não carrega nenhuma BIOS ou firmware. Nosso ELF é carregado diretamente.
`-kernel clock.elf`	Carrega o binário como kernel: o QEMU o coloca em 0x80000000 e salta para o ENTRY.
`-serial mon:stdio`	Conecta o monitor interno do QEMU ao stdio, habilitando Ctrl-A X para encerrar.
`dump: objdump -d -M no-aliases`	Gera o disassembly com nomes reais (sem pseudoinstruções). Útil para debug.

5. Arquivo Dockerfile

Define o ambiente de build e execução de forma reproduzível. Qualquer máquina com Docker executa o projeto da mesma forma, independentemente do sistema operacional host:

Instrução / Diretiva	Explicação
`FROM ubuntu:22.04`	Imagem base: Ubuntu 22.04 LTS. Fornece o apt e o sistema base.
`ENV DEBIAN_FRONTEND=noninteractive`	Desabilita prompts interativos do apt durante a instalação de pacotes.
`gcc-riscv64-unknown-elf`	Toolchain GCC completo para compilar Assembly RISC-V em um host x86-64.
`qemu-system-misc`	Pacote que contém qemu-system-riscv32, o emulador da plataforma virt.
`&& rm -rf /var/lib/apt/lists/*`	Remove o cache do apt para reduzir o tamanho da imagem Docker.
`WORKDIR /project`	Define o diretório de trabalho dentro do container.
`COPY clock.S link.ld Makefile .`	Copia apenas os 3 arquivos fonte necessários para dentro do container.
`RUN make all`	Compila durante o build da imagem. Se o código tiver erros, o build falha aqui.
`CMD ["make", "run"]`	Comando padrão ao iniciar o container: executa o QEMU com o binário compilado.